

Algebra Computation Practice Quiz 3a

a) The underlying problem or motivation of polynomial interpolation is to come up with a function that passes through given data points. Such a function can then be used for differentiation or integration. Also, the resulting polynomial can be used to estimate intermediate values between nodes. Polynomial interpolation is dependent upon the aspects of uniqueness and existence. If each data point $(x_i, f(x_i))$ is distinct, then there is a unique interpolating polynomial of degree up to n passing through each data point. The fundamental theorem of algebra also comes into play. If we let $p(x)$ be a polynomial of degree $\leq n$ w/ $p(x_i) = f(x_i) \forall x_i$. Assuming p and q are not identical, we then let $r(x) = p(x) - q(x)$, with r degree $\leq n$. $r(x)$ has $n+1$ distinct roots x_i , implying $r \equiv 0$ (more roots than degree), thus implying $p - q \equiv 0 \iff p = q$, a contradiction to the previous assumption. It follows that a polynomial of degree n has exactly n roots, possibly repeated, possibly complex. The root above has more than n roots, though.

Existence is an aspect that defines the interpolation using Lagrange polynomials, serving as "dummy unknowns" or slots. That is, each data point has a corresponding polynomial which equals 0 when x is a value other than the value for which it is defined. The overall polynomial is defined as the sum of these Lagrange polynomials. More formally, we let $L_j(x) = \prod_{i \neq j} \frac{x - x_i}{x_j - x_i}$. So $p_n(x) = \sum_{j=0}^n y_j L_j(x)$.

One of the key assumptions of interpolation is that the data points are exact, i.e., the functional values $f(x_i)$ can be obtained without any error. This assumption may not always hold in practice, however, potentially decreasing interpolation accuracy & usefulness.

b) There are several methods for computing interpolating polynomials, arranged in a table along with their respective advantages & disadvantages.

Strategy	Summary	Pros	Cons
Matrix (Lagrange + linear system)	$p_n(x) = a_0 + a_1x + \dots + a_nx^n$ $p_n(x_i) = a_0 + a_1x_i + \dots + a_nx_i^n = y_i$	• directly solve for coefficients • easy to set up	• large values of n require large matrix for large n , increase error?
Lagrange	$L_j(x) = \prod_{i \neq j} \frac{x - x_i}{x_j - x_i}$	• easily calculatable	• not updatable for large n • computationally intensive
Newton Divided Difference	direct difference method to compute coefficients	• updatable • simple computing • efficient	• potential instability in implementation
Computer	using pre-defined software functions	• very quick	• may not produce explicit function for analysis

The usefulness of all strategies depends on the problem. Lagrange is possibly the simplest to implement, but is not updatable with larger values of n , (i.e., it needs to be calculated from scratch again). Newton Divided Difference is updatable & efficient, but may not work well with highly oscillatory data.

c) $\forall x_i \in [a, b]$ distinct, $f \in C^m$ on $[a, b]$, $x \in [a, b]$, then $f(x) = p_n(x) + R_n(x)$ where $R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x-x_0) \dots (x-x_n)$. We expect the error $\rightarrow 0$ as $n \rightarrow \infty$ with a well-behaved function, because $f^{(n+1)}$ would be uniformly bounded (i.e., error can't increase).